AI-Assisted Image Detection for Mitigating Airborne Collisions

ECE 6380 AI Hardware Course Project Report

Jordan Capelle
Karl Garman
Mohomad Sabur

Contents

Abstract

A compact computer (Raspberry Pi5™) is small and light enough to be used on small Unmanned Aircraft Systems ("sUAS", defined as UASs weighing under 55Lb.). The Pi5 is trained to recognize images of birds. With sufficient accuracy and speed in identifying birds from an on-board camera, positive identifications of birds can be used to produce steering commands so the sUAS can avoid colliding with the birds. This reduces the likelihood of an expensive piece of equipment being downed by bird strikes and also mitigates hazards to people and property on the ground from post-collision falling debris. Quantitative accuracies (%) of the trained Pi5 are

reported, as well as the time (seconds) to produce a bird classification. These are reported for two presumably identical pieces of hardware (Pi5 with the "AI HAT+" neural network accelerator with 13 tera-operations per second "TOPS") and compared the benchmark of a third Pi5 as a control which lacks the AI HAT+ add-on.

Introduction

Microelectromechanical systems (MEMS) have enabled UASs to become significantly smaller and more capable over the past two decades. Legacy radio-controlled aircraft were the realm of hobbyists, operated within line of sight of the user, and required pilot-like skill to fly. Now, MEMS-enabled sUAS are operated by people with no aviation background, and operations beyond visual line of sight (BVLOS) are becoming more common.

A challenge with sUAS operated BVLOS has to do with collision avoidance. The key challenge to integrating these vehicles into the National Airspace System involves the safety impact of these vehicles, both the airborne collision hazard (colliding with other aircraft) and the ground collision hazard (colliding with people or objects on or affixed to the ground). Avoidance of other aircraft has traditionally been done by see-and-avoid methods, where a pilot uses their own eyes to see other aircraft and perform steering maneuvers to avoid a collision. In areas of reduced visibility, pilots could use radar and synthetic vision, supplemented by air traffic control assistance, to avoid collision hazards with other aircraft. Likewise, with sUASs operating within visional line of sight of the operator, that operator would be expected to keep the sUAS clear of other aircraft and of ground-based obstacles (e.g., towers, houses, trees). With the proliferation of sUAS operating BVLOS, however, onboard video cameras which are relayed in real time can be used to identify collision hazards within the camera's field of view.

Technical Challenge – Mitigating Collision Hazards of Small Unmanned Aircraft Systems

We now develop the rationale for identifying birds using AI-enabled onboard video on sUAS. But first, a discussion of the current practices for mitigation collisions with other airborne objects (aircraft) and with ground-based objects (terrain and structures) is in order.

For ground collision hazards, aviation maps and electronic databases exist that show the locations, maximum heights, and shapes of terrain features and human-created structure. The

ground collision hazard can be mitigated by simply keeping the sUAS's flight path away from such terrain and structures.

Most airborne collision hazards that can be mitigated by controlling the sUAS's altitude. Federal regulation, 14 CFR 91.119 -- Minimum safe altitudes: General[1], limits the minimum altitudes for most aircraft operations. This is typically a minimum of 500 feet above the surface of the earth, and no less than 500 feet from any person, vessel, vehicle, or structure. These operating rules, called "Part 91" due to the federal regulatory rule they fall under, evolved presuming that all aircraft were human occupied. The growth of sUAS led to a new regulatory rules and guidance, namely "Part 107 – Small Unmanned Aircraft Systems" and Chapter 11 "Unmanned Aircraft Systems (UAS)" of the Aeronautical Information Manual[2]. The rule which governs maximum altitudes for sUASs is governed by 14 CFR 107.51 -- Operating limitations for small unmanned aircraft[3], no higher than 400 feet above the surface, or within a 400 foot radius of a structure, or no higher than 400 feet above that structure's highest point. By comparing the operating rules for [typically human-occupied aircraft] in Part 91 with the sUAS operating rules in Part 107, we see that the airspace was effectively segregated to mitigate collision hazards between these kinds of aircraft operations. Even then, an increasing number of sUASs are networked and remotely identifiable. This type of system can enable mutual collision avoidance among sUASs, even those which are operating in the same altitude block (i.e. surface to 400 feet). So, these kinds of airborne collisions can typically be mitigated by (1) keeping different operating altitudes between human-occupied aircraft and sUASs, and (2) network-enabled situational awareness among sUASs operating in the same altitude block.

[1] https://www.ecfr.gov/current/title-14/chapter-I/subchapter-F/part-91/subpart-B/subject-group-ECFRe4c59b5f5506932/section-91.119
[2] https://www.faa.gov/air_traffic/publications/atpubs/aim_html/chap_11.html
[3] https://www.ecfr.gov/current/title-14/chapter-I/subchapter-F/part-107/subpart-B/section-107.51

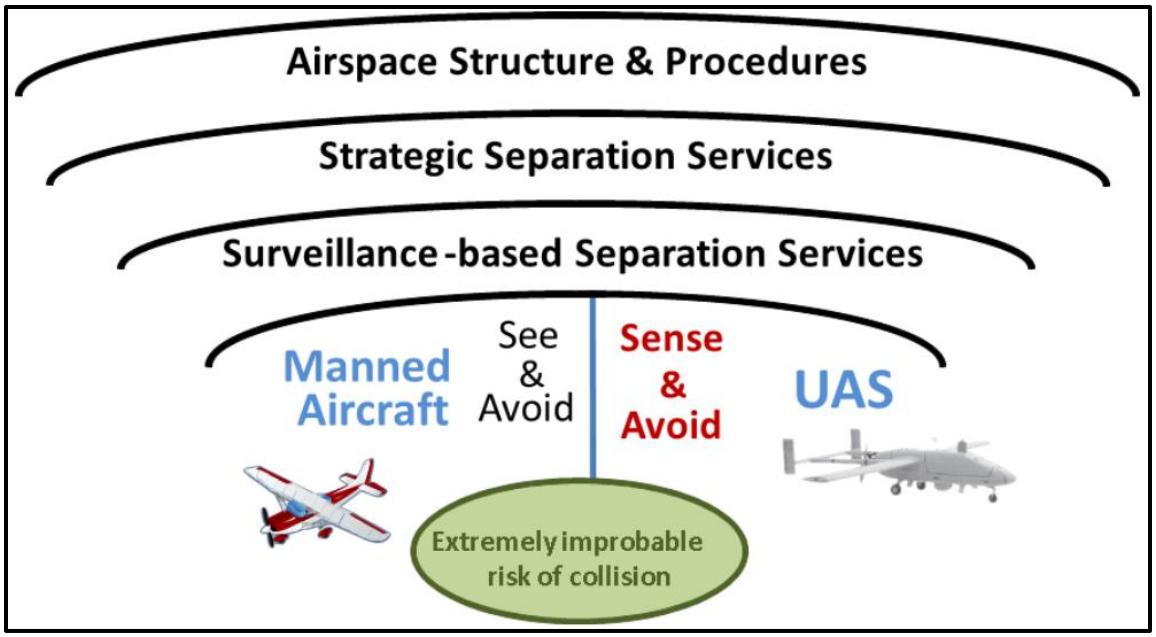

Figure 1. A Layered Approach for Collision Avoidance
(source: FAA Aeronautical Information Manual, Ch.11)

Problem Statement

This leaves the issue of collision hazards that we cannot control. Namely, what objects in the air and on the ground are not going to be in known locations or can be commanded to steer away from the flight path of an sUAS. Let us assume that ground-based moving objects will not be high enough to cause a collision hazard to sUASs, unless those sUASs are already flying at extremely low altitude (like the height of a truck or mobile crane). This leaves unpredictable flying objects as the remaining category of collision hazards. Nearly all insects are too small and lightweight to down a sUAS. However, birds are unpredictable, have no regard of federal aviation regulations, and are often large enough to cause disabling damage to a sUAS on impact.

Thus, the remaining unmitigated airborne collision hazards to sUAS come from birds. Such an impact then creates results in both falling sUAS wreckage and a bird carcass, which can cause hazards to people and property on the ground. If one mitigates the bird collision hazard by identification of birds in real-time and steering away from them, then this collision-induced ground collision hazard is mitigated as well. The feasible identification of an airborne bird collision hazard, through real-time AI-assisted visual means, is the motivator of this work.

Literature Survey

A review of the published literature showed that recent works tended into two categories – using machine vision and machine learning to create new vehicle-focused capabilities for sUASs, and to use AI-enabled sUASs to gather knowledge about the natural environment. This is important, because our work operates at the nexus of these two bodies of work.

Gerardi (2020)[4] utilized image data from infrared sensors and fed them into a feed forward convolutional neural network (CNN). This CNN classified the images as either having UASs or not. The U.S. Army is naturally interested in using infrared cameras in detecting potential adversary UASs, so their use cases focused on that threat scenario (vice avian). Recommendations which came from this work included exploring other machine learning methods such as object detection and the "You Only Look Once" (YOLO) algorithm.

Di Cecio, et.al. (2024)[5] also looked at the UAS detection topic. This work explored using edge computing to classify sUASs, tuning a deep learning model with two different YOLO algorithms (v3 and v8), and implementing this on a Field Programmable Gate Array (FPGA). The FPGA was deemed desirable because of its architectural features (parallelism, high speed, low power, low latency). This system was implemented System-on-Chip (SoC) and trained on a dataset of sUAS images at various distances. Latency was a key consideration of the work, as it is important to the feasibility of on-board AI image processing on a sUAS.

Schermann, et.al. (2023)[6] developed a proof-of-concept deep learning-based system to detect the presence of UASs, also with a security mindset. Where this wok differed from Gerardi and Di Cecio is that Schermann used networked approach to create an "intrusion detection system." This was more of an effort to create a defensive paradigm for a facility from a "threat" UAS, rather than to identify and avoid collision with another flying vehicle. Yet, this work developed a machine learning system and trained on a dataset of "realistic cyber defense scenarios." So, while the work differed from our proposed use case, the nexus with detecting a small flying vehicle is intriguing.

---

[4] Gerardi, "*The Applicability of Machine Learning Methods on Infrared Video Data*", U.S. Army Combat Capabilities Development Command Armaments Center, 2020.

[5] Di Cecio, et.al., "*On-board drone classification with Deep Learning and System-on-Chip implementation*", J. Phys.: Conf. Ser. **2716** 012059, 2024.

[6] Schermann, et.al.,. "Risk-Aware Intrusion Detection and Prevention System for Automated UAS", 2023 IEEE 34th ISSREW, 2023.

Ullah (2024)[7] addressed obstacle detection in the navigation path of sUASs and focused on flight path planning. Their work used CNN-based object detection algorithms, particularly the YOLO family, deployed on edge devices including the Raspberry Pi. As their work focused on obstacle identification and avoidance, both speed and accuracy were experimentally evaluated. Their findings highlighted the feasibility of integrating CNN-based vision systems with small-board computer processing for sUAS applications in dynamic environments. Their most promising results were with the YOLOv6, though subsequent versions of YOLO now exist.

Gallagher, et.al. (2024)[8] provided a survey of YOLO algorithms, focusing on multispectral imaging and deep learning to support diverse use cases. It surveyed the use of these technologies to advance object detection and classification, reviewing 400 total papers and 200 of them in detail. This work also noted that UAS for YOLO multispectral applications have doubled since 2020, validating the growth of this new field. The plurality of sensor fusions in these papers were red-green-blue with long-wave infrared, and our work principally focused on the visible light spectrum. Nonetheless, this work gives inspiration that object detection and classification can extend beyond the visible spectrum of light and the image databases which were captured in the visible spectrum.

A variety of surveyed literature focused on detecting and classifying natural living objects using techniques employing sUAS-mounted cameras. Arriola-Valverde et.al., (2023)[9] used CNN-based deep learning techniques on the YOLO framework to automatically detect coffee plants. The applications of this work focused on applications such as crop inventories, containment of crop diseases, and farm management. Their results showed mean accuracies in the 90+ plus range. Though this work overcame challenges of classifying coffee crops from among other plant life using red-green-blue visible light cameras, the processing time had the luxury of being unconstrained (unlike collision avoidance applications).

---

[7] Ullah, et.al., "*Autonomous Control with Vision and Deep Learning: A Raspberry Pi Edge Computing Platform for Obstacle Detection in SUAV Path*", 7th International CommNet Conference, 2024.
[8] Gallagher, et.al., "*Surveying You Only Look Once (YOLO) Multispectral Object Detection Advancements, Applications, and Challenges*", IEEE Access, *DOI 10.1109/ACCESS.2025.3526458, 2024.*
[9] Arriola-Valverde, et.al., "*Coffee Crop Detection from UAS Orthomaps with Convolutional Neural Networks*", 2023 IEEE Conference on Agrifood Electronics, 2023.

Likewise, Tin, et.al., (2024)[10] also looked at crop and weed identification for agriculture, using a deep learning model and images captured by sUAS as an "alternative" to a YOLO model. The authors' approach employed Region-based Convolutional Neural Networks (RCNN) for extracting features with the Support Vector Machine (SVM). This combination differs from YOLO architecture because it uses a region-based technique in image processing. This RCNN-SVM divides the region proposals (bounding boxes) into several groups and fine-tunes the bounding box coordinates of those regions that are more likely to contain items of interest (weeds). This technique resulted in a 98% accuracy score. Like the coffee crop study, however, the luxury of processing time abounded.

Lawrence, et.al., (2023)[11] describes an environmental sensing application which directly focused on classifying the living quarters of a particular bird species. This UAS-based detection system used YOLOv4-tiny and YOLOv5n architectures for training and inferencing aerial photos of vegetated study areas to identify these small objects. The best performing model generated a "mean average precision" (mAP) of 95%, while maintaining an inference speed of 2.5 frames per second. While this capability is laudable, the analysis and classification were done by processing them after the flight, employing model development techniques such as tilting and sliced aided hyper inferencing. This is not feasible for a collision avoidance algorithm looking at a possible collision hazard – it must be classified on-the-fly. So, while the environmental applications of sUAS-based image classifications were successful in their intentions, analysis time was an abundant resource.

Having explored the literature to this point, it became apparent that the recent peer-reviewed articles and conference papers tended towards increasing sUAS capabilities and environmental sensing, both through onboard vision and image classification. We refined our literature survey to focus specifically on bird detection and were able to locate a couple such recent papers. Alqaysi, et.al. (2021)[12] described the problem for surveillance of birds against a sky, as the objects are small in a large volume and a constantly changing background which requires high

---

[10] Tin, et.al., "*Exploring an Alternative Approach: RCNN-SVM for Weed Identification in Aerial images Captured by UAS*", 2024 International Conference on Emerging Trends in Networks and Computer Communications, 2024.

[11] Lawrence, et.al., "*UAS-Based Real-Time Detection of Red-Cockaded Woodpecker Cavities in Heterogeneous Landscapes using YOLO Object Detection Algorithms*", Remote Sens. 15, 883, 2023.

[12] Alqaysi, et.al., "*A Temporal Boosted YOLO-Based Model for Birds Detection around Wind Farms*", Journal of Imaging, 7, 227, 2021.

resolution frames. Excitingly, the use case did involve detecting birds to prevent collisions, specifically birds colliding with wind turbines. This paper used a YOLOv4-based model in grayscale video and two datasets, one to train the model and the other introducing tilting to improve small bird detection. These models were then both utilized to set up a third "ensemble" detector, which further improved the mAP to range between 82% and 90%. The author's use case postulated that improving object detection accuracy could (1) mitigate avian mortality by enabling more suitable location choices for these turbines, and (2) to improve the collision avoidance systems used in wind energy facilities. So, this paper's work was at the nexus of technical capability and environmental sensing, with bird collision avoidance as a prime purpose.

Rounding out our literature survey, Sun, et.al. (2025)[13] introduced and tailored a flying bird dataset "FBD-SV-2024" for evaluation of flying bird detection algorithms in surveillance videos, comprising 28K+ frames from realistic surveillance scenarios. The authors articulate the unique challenges and inherent value of having such a flying bird dataset available. For instance, birds can exhibit inconspicuous features in some frames and not in others, they have generally small sizes in a camera's instantaneous field of view, and they vary their shapes during flight. The authors also describe experimentation of video object detection algorithms and identify current challenges for using bird detection algorithms on such data. Laudably, the authors used 12 different object detection algorithms such as YOLOv5L through v10L, et.al., and reported quantitative experimental results from each (such as detection performance and the detection speed). So, this paper gave a generally good overall sense of this specific problem, though the images appeared to be taken from stationary cameras, and not moving sUAS platforms.

A gap in the literature appeared, which was an opportunity for our team to explore and contribute to the body of knowledge and develop a new capability. None of the reviewed papers used a YOLO model on an AI chip and applied it to the use case of identifying avian flight hazards. Such real-time bird identification capability could be employed for giving flight control commands to an sUAS using an autopilot, or resolution advisories to an sUAS using a human in-the-loop remote pilot. Also, using an onboard AI-driven bird identification capability would relieve the need for time-consuming communication to a cloud-based decision-making paradigm. The Hailo-8L "M.2 AI HAT" (13 TOPS) processor was designed to be mated with the Raspberry

[13] Sun, et.al., "*FBD-SV-2024: Flying Bird Object Detection Dataset in Surveillance Video*", www.nature.com/scientificdata/, *12:530* | https://doi.org/10.1038/s41597-025-04872-6, 2025.

Pi5, and thus was chosen for this work. This hardware was also a natural complement to the literature surveyed for this work, as various published works had used the Raspberry Pi single-board computer.

Experimental Setup, Benchmarking, and Methodology

To assess the performance of the Raspberry Pi5 for onboard bird detection, a standardized set of baseline tests were conducted for the Raspberry Pi5 alone and the Raspberry Pi5 with the AI Hat. Due to differences in individual software implementations, each member retained their own functioning YOLO-based detection code; however, all evaluations were performed under a common experimental setup to ensure comparability. The objective of these evaluations was to quantify detection accuracy, responsiveness, robustness, and operational stability under controlled and repeatable conditions that approximate the sensing constraints of a small unmanned aircraft system.

Hailo-8L Overview

The Raspberry Pi AI HAT+ used in this study incorporates the Hailo-8L neural processing unit (NPU), a specialized edge-AI accelerator designed to augment the general-purpose CPU of the Raspberry Pi5.[14] NPUs are optimized for the tensor and matrix operations that dominate modern neural-network inference, enabling significantly higher throughput and lower latency than CPU-only execution.[15] In this project, inference on the Raspberry Pi5's ARM Cortex-A76 cores is constrained by their general-purpose, sequential execution model and limited efficiency for large-scale tensor operations.[16] Offloading neural-network workloads to the Hailo-8L therefore enables real-time object detection in scenarios where CPU-based processing would otherwise introduce reduced frame rates or increased latency. [17]

---

[14] Raspberry Pi AI Hat+, Raspberry Pi Ltd, Oct. 2024, datasheets.raspberrypi.com/ai-hat-plus/raspberry-pi-ai-hat-plus-product-brief.pdf.
[15] Gaming, Corsair. "CPU vs GPU VS NPU: What's the Difference?" CORSAIR, 4 Mar. 2025, www.corsair.com/us/en/explorer/diy-builder/power-supply-units/cpu-vs-gpu-vs-npu-whats-the-difference/?srsltid=AfmBOooXjdJUqLlGbPk_qghF6D_8V6YcjYPYPD-feldGEHTd7wwWxyxb&utm_source=chatgpt.com.
[16] Raspberry Pi5, Raspberry Pi Ltd, Dec. 2025, pip.raspberrypi.com/documents/RP-008348-DS-3-raspberry-pi-5-product-brief.pdf.
[17] Gaming, Corsair. "CPU vs GPU VS NPU: What's the Difference?" CORSAIR, 4 Mar. 2025, www.corsair.com/us/en/explorer/diy-builder/power-supply-units/cpu-vs-gpu-vs-npu-whats-the-

The Hailo-8L is designed by Hailo Technologies and fabricated by TSMC using a 16-nanometer process node, a mature manufacturing technology well suited for low-power embedded systems.[18] [19] The accelerator provides up to 13 trillion operations per second (13 TOPS) of neural-network inference performance while typically consuming approximately 1.5 watts of power. [20] Integration with the Raspberry Pi5 is achieved over a PCIe Gen 3 interface, allowing efficient data transfer between the host and the accelerator. [21] In the context of this research, the inclusion of the Hailo-8L improves per-frame inference time and effective frame rate, reducing missed detections and increasing reliability for real-time bird detection using an onboard camera.[22] This CPU-NPU combination provides a compact and power-efficient platform suitable for bench-level collision-avoidance research relevant to small unmanned aircraft systems. [23]

AI Model Integration

One shortcoming of the Hailo-8L processor is that to run an AI model, the model has to be converted to a Hailo Executable File or .hef file. The .hef file is a proprietary file type that is exclusive to the Hailo architecture. The model we selected for our project was the YOLO model which is natively built on PyTorch. (Ultralytics, n.d.). The native PyTorch or .pt file will not work directly with the Hailo chip without conversion.

File conversion is accomplished by using the Hailo Dataflow Compiler tool or DFC. The DFC conversion software cannot run on the Pi5 due to its ARM 64 processor type. A separate PC

---

difference/?srsltid=AfmBOooXjdJUqLlGbPk_qghF6D_8V6YcjYPYPD-feldGEHTd7wwWxyxb&utm_source=chatgpt.com.

[18] Maxfield, Max. "Are These the Top-Performing Edge AI Processors?" *EEJournal*, Techfocus Media inc, 1 Sept. 2023, www.eejournal.com/article/are-these-the-top-performing-edge-ai-processors/?utm_source=chatgpt.com.

[19] "Technology Node." *WikiChip*, WikiChip, 5 Oct. 2025, en.wikichip.org/wiki/technology_node.

[20] "8L Entry-Level AI Accelerator Solutions." *Hailo*, Hailo Technologies Ltd, 19 Oct. 2025, hailo.ai/products/ai-accelerators/hailo-8l-ai-accelerator-for-ai-light-applications/#hailo8l-overview.

[21] *Raspberry Pi AI Hat+, Raspberry Pi Ltd, Oct. 2024, datasheets.raspberrypi.com/ai-hat-plus/raspberry-pi-ai-hat-plus-product-brief.pdf*.

[22] *Raspberry Pi AI Hat+*, Raspberry Pi Ltd, Oct. 2024, datasheets.raspberrypi.com/ai-hat-plus/raspberry-pi-ai-hat-plus-product-brief.pdf.

[23] Gaming, Corsair. "CPU vs GPU VS NPU: What's the Difference?" *CORSAIR*, 4 Mar. 2025, www.corsair.com/us/en/explorer/diy-builder/power-supply-units/cpu-vs-gpu-vs-npu-whats-the-difference/?srsltid=AfmBOooXjdJUqLlGbPk_qghF6D_8V6YcjYPYPD-feldGEHTd7wwWxyxb&utm_source=chatgpt.com.

using x86 processor architecture is required to use the DFC. The conversion process consists of four steps. Step one, the native YOLO .pt file is converted to ONNX using Python instructions provided by Ultalytics. The second step is to use the DFC to convert ONNX models to a Hailo Archive or .har file. The third step is to quantize the .har file to the .har quantized format. Finally, the fourth step converts the .har quantized file to the usable .hef or Hailo Executable File.

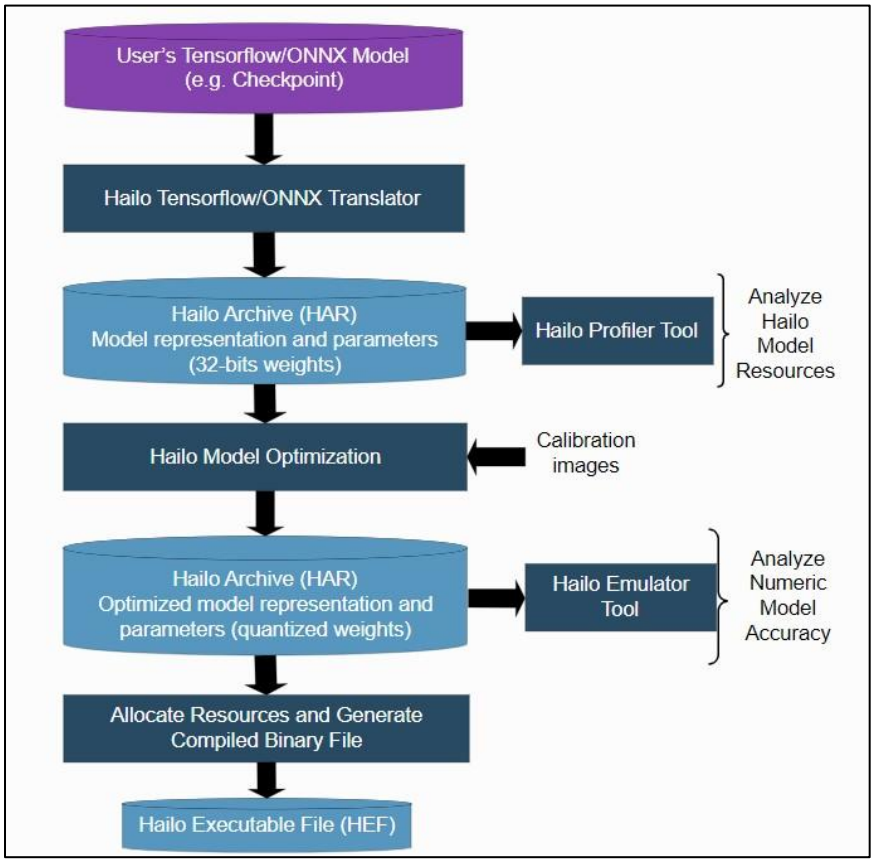

Figure 2. Model build process (source: Hailo Dataflow Compiler User Guide, Release 3.27.0, 26 March 2024).[24]

Experimental Test Setup

[24] The Hailo Dataflow Compiler toolchain enables users to generate a Hailo executable binary file (HEF) based on input from at Tensorflow checkpoint, a Tensorflow frozen graph file, a TFLite file, or an ONNX file. The build process consists of several steps including translation of the original model to a Hailo model, model parameters optimization, and compilation. This process writeup is from Ultralytics.

*Figure 2. Selected Ground Truth Reference Images. Upper left to lower right: Slide 2 "*Single bird, medium distance*", Slide 6 "*Cluttered background (trees)*", Slide 13 "*Dense trees*", Slide 17 "*Kite*".*

In lieu of a drone-mounted platform, a bench-level approximation was used wherein each Raspberry Pi's camera was directed toward a secondary display (monitor or laptop). This display presented a predetermined sequence of still images and video clips of birds and non-birds. This served as a reproducible stimulus source for the image-recognition algorithms running on each Pi, regardless of local implementation differences. The content of the slideshow was jointly defined by the team and constituted the common "ground truth" input set for all evaluations.

Each item in the slideshow—whether still image or video—was assigned a unique identifier. The team also annotated whether each item contained a bird, as well as any relevant scene characteristics (e.g., cluttered background, low light, small or distant target). Establishing this ground truth allowed direct comparison between the detector's output and expected classifications.

Table 1 — Slideshow Ground Truth Reference

Sample ID	Type	Bird Present (Y/N)	Notes (Scene Description)
1	Image	Y	Single bird, close range, clear lighting
2	Image	Y	Single bird, medium distance
3	Image	Y	Single bird, distant
4	Image	Y	Multiple birds
5	Image	Y	Partially occluded bird (branches, leaves)

6	Image	Y	Cluttered background (trees)
7	Image	Y	Bird against plain sky
8	Image	Y	Low-light bird
9	Image	Y	Bird slightly blurred (motion)
10	Image	Y	Bird camouflaged
11	Image	N	Clear sky
12	Image	N	Cloudy sky
13	Image	N	Dense trees
14	Image	N	Urban scene
15	Image	N	Shoreline
16	Image	N	Horizon
17	Image	N	Kite
18	Image	N	Human figure
19	Image	N	Airplane
20	Image	N	Drone
21	Video	Y	Bird in slow, steady flight across the sky
22	Video	Y	Flock of birds with varied movement
23	Video	Y	Bird obscured by forest flying away
24	Video	N	Moving clouds
25	Video	N	Moving through trees

Test Procedures

Four categories of evaluations were carried out. These were designed to characterize (1) accuracy under controlled conditions, (2) behavior when presented with moving stimuli, (3) sensitivity to viewpoint and display variations, and (4) operational stability during extended operation.

1. Still-Image Accuracy Evaluation

The first test quantified the detector's performance when presented with static imagery. Sample ID's 1-20 were shown on the monitor for five seconds while the Pi's camera captured the screen and the YOLO model produced classification outputs. For each sample, detections were marked as:

- True Positive (TP): a bird was present and detected.
- False Negative (FN): a bird was present but not detected.
- False Positive (FP): a bird was detected when none was present.
- True Negative (TN): no bird was present, and none was detected.

Precision (TP / (TP + FP)) and recall (TP / (TP + FN)) were computed from the aggregate counts.

Table 2 — Still Image Accuracy Summary

Tester Name	Platform	Total Samples	TP	FN	FP	TN	Precision	Recall	Notes
Jordan	Pi5	20	9	1	0	10	1	0.9	Failed to detect the partially occluded bird
Mohomad	Pi5 + AI	20	10	0	1	9	0.9	1	Sample ID 17 (Kite) was identified as a bird with 25% confidence

2. Video-Based Detection Performance

The second test examined performance when the target object was moving. This more closely resembles the operational scenario of an sUAS encountering birds in flight. Team members played sample ID's 21-25 with each video clip full-screen while the detector processed the live camera feed directed at the monitor. Each video is set to a duration of approximately 10 seconds. Evaluators noted:

- Whether detections were consistently produced during segments containing birds.
- Whether false positives occurred during bird-absent segments.
- The qualitative smoothness of inference (e.g., smooth, acceptable, choppy).
- Approximate/average frame rates when available.

Table 3 — Video Detection Summary

Tester Name	**Platform**	**Video ID**	**Bird Present (Y/N)**	**Reliable Detection (Y/N)**	**False Positives (Y/N)**	**Smoothness**	**Approx. FPS**	**Notes**
Jordan	Pi5	21	Y	Y	N	Normal	3.23	
	Pi5	22	Y	N	N	Normal	3.40	
	Pi5	23	Y	N	N	Normal	3.37	
	Pi5	24	N	Y	N	Normal	3.38	
	Pi5	25	N	Y	N	Normal	3.29	
Mohomad	Pi5+AI	21	Y	Y	N	Smooth	30	
	Pi5+AI	22	Y	Y	N	Smooth	30	
	Pi5+AI	23	Y	Y	N	Smooth	30	
	Pi5+AI	24	N	Y	N	Smooth	30	
	Pi5+AI	25	N	Y	N	Smooth	30	

3. Robustness Under Varying Presentation Conditions

To approximate less ideal sensing environments, the third test varied the presentation conditions. Sample ID's 3 (single bird, distant), 5 (partially occluded bird), 8 (low-light bird), 11 (clear sky), 13 (dense trees),19 (airplane), and 20 (drone) were played for a 5 second duration. Three representative degradations were tested:

- Increased viewing distance, reducing apparent bird size.
- Reduced display brightness, approximating low-light scenes.
- Oblique camera angles, simulating off-axis viewpoints. For each condition, a smaller subset of images (containing both bird and non-bird cases) was presented. Results were again logged as TP, FN, FP, and TN for each condition.

Table 4 — Robustness Evaluation Summary

Tester Name	Platform	Total Samples	TP	FN	FP	TN	Precision	Recall	Notes
Jordan	Pi5	7	2	1	0	4	1	0.67	Failed to detect the partially occluded bird (ID 5)
Mohomad	Pi5 + AI	7	3	0	0	4	1	1	

4. Short-Term Stability and Responsiveness

The fourth evaluation assessed the resilience of the system during continuous operation. A looping sequence of samples was presented for a period of approximately ten minutes. Testers monitored:

- Whether the detector crashed or terminated unexpectedly,
- Whether performance degraded noticeably over time,
- Whether detections remained responsive and timely throughout the test.

Table 5 — Stability and Responsiveness

Tester Name	Platform	Duration (min)	Crashes (Y/N)	Slowdowns (Y/N)	Consistent Detections (Y/N)	Peak Temperature	Notes

Jordan	Pi5	10	N	N	Y	123°F	Had the same failed detections as previous tests
Mohomad	Pi5 + AI	12	N	N	Y	152°F	

Relation to Performance Objectives

These baseline procedures map directly to the quantitative objectives of this study. Real-time performance and detection latency are reflected in the frame-rate data and qualitative smoothness assessments. Detection accuracy is derived from the aggregate precision and recall values across both static and dynamic tests. Robustness is evaluated through systematic variation of display conditions, and system reliability is demonstrated through continuous-operation testing. Collectively, these evaluations provide a comprehensive assessment of the Raspberry Pi platform's suitability for onboard bird-detection tasks in support of collision-avoidance systems for small unmanned aircraft.

Results & Discussion

Experimental Results Overview

The performance of the bird-detection system was evaluated using four complementary tests: still-image classification, video-based detection, robustness under challenging visual conditions, and long-duration stability. Each test was conducted using the same physical setup and input stimuli on two platforms: a Raspberry Pi5 operating in a CPU-only configuration, and a Raspberry Pi5 equipped with the Hailo-8L AI accelerator via the Raspberry Pi AI HAT+. This approach enabled a controlled comparison of inference accuracy, throughput, and system behavior under identical conditions.

Still Image Classification Performance

In the still-image classification test, both platforms demonstrated strong baseline detection capability. The Raspberry Pi5 operating without acceleration, correctly identified the majority of bird images while producing no false positives. However, one false negative occurred

when the bird was partially occluded, indicating a limitation in recall under more challenging visual conditions.

The AI-accelerated configuration achieved perfect recall across the test set, successfully detecting all birds presented. One false positive was observed when a kite was misclassified as a bird with low confidence. This behavior suggests that while the accelerated model is more sensitive and less prone to missed detections, it may be more susceptible to visually similar non-bird objects. In the context of collision avoidance, this tradeoff favors recall over precision, as missed detections pose a greater safety risk than occasional false alerts.

Video Detection and Real-Time Performance

The most significant performance difference between platforms was observed during video-based detection. The CPU-only Raspberry Pi5 operated at approximately 3–4 FPS during live video inference. While detections were mostly correct, this frame rate is insufficient for time-critical applications such as collision avoidance, where fast-moving objects must be identified and reacted to within fractions of a second.

In contrast, the Raspberry Pi5 equipped with the Hailo-8L consistently achieved approximately 30 FPS. Detection was smooth, responsive, and visually stable across all tested videos. This improvement represents nearly an order-of-magnitude increase in throughput and demonstrates the effectiveness of offloading neural-network inference from the general-purpose CPU to a dedicated NPU. The results confirm that hardware acceleration is essential for achieving real-time perception on embedded platforms.

Robustness to Challenging Visual Conditions

Robustness testing further highlighted the limitations of CPU-only inference. When presented with partially occluded birds, the Raspberry Pi5 continued to miss detection, reducing recall. These cases are particularly relevant to real-world sUAS operation, where birds may appear briefly, at a distance, or against complex backgrounds.

The AI-accelerated system successfully detected all birds in the robustness test set, maintaining both precision and recall. This suggests that the increased inference capacity and parallelism of the Hailo-8L enable more reliable feature extraction under degraded visual conditions, improving the system's ability to generalize beyond ideal inputs.

Stability and Thermal Behavior

Both platforms demonstrated stable operation during extended runtime tests, with no crashes or significant slowdowns observed. The AI-accelerated configuration operated at higher temperatures than the CPU-only system, as expected given the additional compute activity. However, temperatures remained within safe operating limits, and no thermal throttling was observed during the test duration.

These results indicate that the Raspberry Pi5, when properly cooled, can sustain long-duration AI inference workloads, including those accelerated by the Hailo-8L, without compromising system stability.

Discussion and Implications for sUAS Applications

Collectively, the results demonstrate that while the Raspberry Pi5 is capable of performing bird detection using CPU-only inference, its performance is insufficient for real-time collision-avoidance applications. The limited frame rate and reduced robustness under challenging conditions constrain its usefulness in safety-critical scenarios.

By contrast, integrating the Hailo-8L AI accelerator transforms the system into a viable real-time perception platform. The substantial improvements in frame rate, recall, and robustness directly support the timing and reliability requirements of sUAS collision avoidance. These findings reinforce the broader conclusion that dedicated edge-AI hardware is not merely an optimization, but a necessity for deploying neural-network-based perception systems on embedded aerial platforms.

Recommendations & Future Work

This project compared a Raspberry Pi5 with and without an “AI HAT”, the latter of which had a 13 TOPS capability. There is a commercially available AI HAT with 26 TOPS capability. A recommendation for follow-on work is to repeat the experiment with a 26 TOPS capability AI HAT and determine the differences in accuracy computation.

This work focused on the ability of algorithms to identify birds. A way to extend this is to include other flying objects, such as aircraft, sUASs, kites, et.al. These are also airborne collision hazards of interest to the sUAS community.

Computation time (the time from image capture to "bird or not bird" determination) is important to collision avoidance work. A high accuracy computation can be functionally worthless if it is not delivered in time to the remote pilot or autopilot, as a tardy resolution advisory would be moot. So, future work can focus on ascertaining and reducing the computation time and producing comparisons of accuracy vs. computation time.

Of course, a series of flight tests with actual sUAS hardware would be desirable. Several representative sUAS models and configurations would be of interest, as the performance of onboard computer/AI systems can depend upon installation variability (e.g., presence/absence of forced air cooling, vibration, electrical interference).

Conclusions

[Karl – add conclusions here, which is a combination of the abstract and the high-level results section, and is concise, a paragraph or two]

References & Resources

Aeronautical Information Manual, Chapter 11, Federal Aviation Administration, accessed November 24, 2025. https://www.faa.gov/air_traffic/publications/atpubs/aim_html/chap_11.html

Alqaysi, et.al., "*A Temporal Boosted YOLO-Based Model for Birds Detection around Wind Farms*", Journal of Imaging, 7, 227, 2021.

Arriola-Valverde, et.al., "*Coffee Crop Detection from UAS Orthomaps with Convolutional Neural Networks*", 2023 IEEE Conference on Agrifood Electronics, 2023.

Code of Federal Regulations, 14 CFR part 91, US Government, accessed November 24, 2025. https://www.ecfr.gov/current/title-14/chapter-I/subchapter-F/part-91/subpart-B/subject-group-ECFRe4c59b5f5506932/section-91.119

Code of Federal Regulations, 14 CFR part 107, US Government, accessed November 24, 2025. https://www.ecfr.gov/current/title-14/chapter-I/subchapter-F/part-107/subpart-B/section-107.51

Di Cecio, et.al., "*On-board drone classification with Deep Learning and System-on-Chip implementation*", J. Phys.: Conf. Ser. **2716** 012059, 2024.

Gallagher, et.al., "*Surveying You Only Look Once (YOLO) Multispectral Object Detection Advancements, Applications, and Challenges*", IEEE Access, *DOI 10.1109/ACCESS.2025.3526458, 2024.*

Gerardi, "*The Applicability of Machine Learning Methods on Infrared Video Data*", U.S. Army Combat Capabilities Development Command Armaments Center, 2020.

Hailo Dataflow Compiler User Guide, Release 3.27.0, 26 March 2024

Lawrence, et.al., "*UAS-Based Real-Time Detection of Red-Cockaded Woodpecker Cavities in Heterogeneous Landscapes using YOLO Object Detection Algorithms*", Remote Sens. 15, 883, 2023.

Schermann, et.al.,. "Risk-Aware Intrusion Detection and Prevention System for Automated UAS", 2023 IEEE 34th ISSREW, 2023.

Sun, et.al., "*FBD-SV-2024: Flying Bird Object Detection Dataset in Surveillance Video*", www.nature.com/scientificdata/, *12:530* | https://doi.org/10.1038/s41597-025-04872-6, 2025.
Tin, et.al., "*Exploring an Alternative Approach: RCNN-SVM for Weed Identification in Aerial images Captured by UAS*", 2024 International Conference on Emerging Trends in Networks and Computer Communications, 2024.

Ullah, et.al., "*Autonomous Control with Vision and Deep Learning: A Raspberry Pi Edge Computing Platform for Obstacle Detection in SUAV Path*", 7th International CommNet Conference, 2024.

Ultralytics YOYO Docs, accessed November 24, 2025. https://docs.ultralytics.com/integrations/onnx/

(end)